QA Automation Interview Study Guide with Code

Java Questions

Difference between List, Set, and Map.

```
List<String> browsers = Arrays.asList("Chrome", "Firefox", "Edge");
Set<String> uniqueIds = new HashSet<>(Arrays.asList("ID1", "ID2"));
Map<String, String> testData = new HashMap<>();
testData.put("username", "qa_user");
testData.put("password", "pass123");
```

Exception handling in automation (DB connection failure).

```
try {
    Connection conn = DriverManager.getConnection(url, user, pass);
} catch (SQLException e) {
    System.out.println("Database connection failed: " + e.getMessage());
    Assert.fail("Test failed due to DB error");
}
```

Difference between == and .equals().

```
String a = new String("test");
String b = new String("test");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Read test data from a file.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}
```

Page Object Model (POM) Login Page Example.

```
public class LoginPage {
    WebDriver driver;

    By username = By.id("username");
    By password = By.id("password");
    By loginBtn = By.id("login");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(loginBtn).click();
    }
}
```

Java 8 streams and lambdas for filtering.

```
List<String> users = Arrays.asList("qa1", "qa2", "dev1");
List<String> qaUsers = users.stream()
                            .filter(u -> u.startsWith("qa"))
                            .toList();
System.out.println(qaUsers); // [qa1, qa2]
```

Reusable helper method example.

```
public void clickElement(By locator) {
    driver.findElement(locator).click();
}
```

Selenium Questions

Explicit wait example.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement el = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("submit")));
```

Handle alert.

```
Alert alert = driver.switchTo().alert();
System.out.println(alert.getText());
alert.accept();
```

Dropdown interaction.

```
Select select = new Select(driver.findElement(By.id("country")));
select.selectByVisibleText("USA");
```

Multiple windows handling.

```
String parent = driver.getWindowHandle();
for (String win : driver.getWindowHandles()) {
    driver.switchTo().window(win);
}
```

Parallel TestNG suite snippet.

```
<suite name="Suite" parallel="tests" thread-count="2">
    <test name="Test1">
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
</suite>
```

Dynamic element locator with XPath.

```
By dynamicId = By.xpath("//button[contains(@id, 'submit')]");
```

SQL Questions

Users registered in last 7 days.

```
SELECT * FROM Users
WHERE registration_date >= CURRENT_DATE - INTERVAL 7 DAY;
```

Find duplicate records.

```
SELECT email, COUNT(*)
FROM Users
GROUP BY email
HAVING COUNT(*) > 1;
```

Second highest salary.

```
SELECT MAX(salary)
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Customers with >5 orders.

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS order_count
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(o.order_id) > 5;
```

Verify UI insert.

```
SELECT * FROM Users
WHERE username = 'qa_user';
```

Users with no orders.

```
SELECT c.customer_id, c.name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

QA Design Questions

Login page test scenarios.

```
- Positive: valid username/password.
- Negative: wrong password.
- Empty fields, SQL injection, special chars.
```

Banking transfer DB checks.

```
- Sender balance debited.
- Receiver credited.
- Transaction record created.
- Balances consistent.
```

Automation vs manual choice.

- Automate repetitive/regression-heavy.
- Keep manual for exploratory.

Smoke vs regression vs sanity.

- Smoke: app launch, login works.
- Regression: old features after release.
- Sanity: quick bug fix validation.

Test data setup and cleanup.

- Insert test data before test.
- Delete/reset after test.

Search box auto-suggest tests.

- Suggestions appear.
- Relevant results.
- Performance.
- Click fills input.